

Pre-building Plan:

Choice of Language:

I will code my implementation in Java. The reasons for this are that despite Python being favoured in building neural networks, due to the wealth of libraries for the language specifically for neural works, for the purposes of this course I wish to reduce reliance in pre-built libraries wherever possible so as to understand the details of how the algorithms work themselves. In addition, I believe an object-oriented approach will be well suited to this project. Having a MLP class with neurons as attributes and algorithms like backpropagation and activation functions as methods of that class seems to me to be a natural fit for this problem. Finally, I will be coding in Java for my upcoming placement, so improving my proficiency in the language is beneficial.

I will code my data cleaning software in Python, as the language has the support of many libraries built for data analysis and management that will make the process as frictionless as it can be. I will be using the Pandas library, as it has support for Excel sheets, allowing me to read and write directly to the .xlsx file.

Cleaning the Data:

I will duplicate the Excel spreadsheet into two files, "Data.xlsx" and "Data Backup.xlsx". The latter contains the original data that will not be modified in case an error is made in cleaning the data I have a backup I can revert to. The former will be the file that will be read from and written to the clean the data. I also will use a text file "Bin.txt" that will contain all modifications of data so we can observe the changes.

```
import pandas as pd

# read excel spreadsheet as data, take columns A to I, set none values as "NA"
df = pd.read_excel('C:/Users/jakub/OneDrive/Documents/Neural Networks/Data.xlsx', na_values=["NA"],
usecols = "A:I")

# print every column in data to observe data types
for column in df.columns:
    print(df[column])
```

```
Name: AREA, Length: 597, dtype: object
Name: BFIHOST, Length: 597, dtype: float64
Name: FARL, Length: 597, dtype: object
Name: FPEXT, Length: 597, dtype: float64
Name: LDP, Length: 597, dtype: float64
Name: PROPWET, Length: 597, dtype: float64
Name: RMED-ID, Length: 597, dtype: float64
Name: SAAR, Length: 597, dtype: int64
Name: Index flood, Length: 597, dtype: float64
```

Observe that the data type for AREA and FARL are "object" instead of "float 64". This introduces the possibility for letters and other non-numeric values to be within those columns. Therefore we must check and remove those characters from the values in those two columns.

Therefore, we write a try, except block to catch any items that cannot be parsed into a floating point.

```
# read excel spreadsheet as data, open bin file
df = pd.read_excel('C:/Users/jakub/OneDrive/Documents/Neural Networks/Data.xlsx', na_values=["NA"],
usecols = "A:I")
bin = open("C:/Users/jakub/OneDrive/Documents/Neural Networks/Bin.txt", "a")

# iterate over AREA rows, try to parse into float, catch non-float values
print("AREA")
for i, row in df["AREA"].iteritems():
    try:
        float(row)
    except ValueError:
        string = "AREA\t" + str(i+2) + "\t" + row + "\n"
        bin.write(string)

# iterate over AREA rows, try to parse into float, catch non-float values
print("\nFARL")
for i, row in df["FARL"].iteritems():
    try:
        float(row)
    except ValueError:
        string = "FARL\t" + str(i+2) + "\t" + row + "\n"
        bin.write(string)
```

Result:

ENTITY	ROW	VALUE
AREA	193	sss
FARL	60	aaa
FARL	382	qqq

Now that we have found all values that are non-numeric and therefore clearly invalid, we simply delete these rows from our data, as there are plenty of other datapoints, and hence interpolating to try and predict what the correct was would yield little difference for a lot of effort. I built and ran code to remove these rows, then verified in the data that the deletion was correct.

```
# remove rows and write back to excel sheet
df = df.drop(df.index[[191, 58, 380]])
df.to_excel('C:/Users/jakub/OneDrive/Documents/Neural Networks/Data.xlsx', index=False)
```

It remains to perform validation on all the other values.

Attribute	Minimum Value	Maximum Value	Justification
AREA	0.00	N/A	Area cannot be negative
BFIHOST	0.00	1.00	Ratio values
FARL	0.00	1.00	Ratio values
FPEXT	0.00	1.00	Ratio values
LDP	0.00	N/A	Distances cannot be negative
PROPWET	0.00	1.00	Ratio values
RMED-1D	0.00	340	Most rainfall in 24 hours (UK)
SAAR	0.00	N/A	Rainfall cannot be negative
Index Flood	0.00	N/A	Value cannot be negative

```
for i, row in df["AREA"].iteritems():
    num = int(round(row))
    if num < 0:
        string = "AREA\t" + str(i+2) + "\t" + str(row) + "\n"
        bin.write(string)
```

Here is an example of how I handle validation of columns (AREA in this example). We check each row value against some condition and write to "Bin.txt" if its met.

ENTITY	ROW	VALUE
AREA	193	sss
FARL	60	aaa
FARL	382	qqq
AREA	588	-999.0
BFIHOST	425	-999.0
FARL	142	994.0
FPEXT	303	-999.0
PROPWET	88	340.0
Index flood	262	-999.0

This is the final state of the "Bin.txt" file. It contains all values that are clearly not numeric as per previous examples, and every numerical value that is clearly of range. Deletion is simple, as per the previous example also. Take not however that previous deletion has changed the row numbers for some of these data points, and therefore would be different to the original data.

In the interest of avoiding hard coding my solution I have automated the cleaning process by introducing validation functions and looping over each column, applying the appropriate function to each column, and dropping rows that fail the validation process.

Standardising and Segmenting the Data:

It then remains to segment data in three subsets: a training subset, a validation subset and testing subset. In order to avoid any seasonal biases that obviously have an impact on rainfall levels, the data should be randomly segmented. However, the catchment data is not sequenced by date, and hence is already random. Therefore, we simply segment the first 60% for training, a subsequent 20% on validation and final 20% on testing.

I will standardise and revert the data with their respective formulae:

$$S_i = 0.8 \left(\frac{R_i - Min}{Max - Min} \right) + 0.1$$

$$R_i = \left(\frac{S_i - 0.1}{0.8} \right) (Max - Min) + Max$$

These formulae can be used on the data by writing functions that compute S_i and R_i values for each row. Then we can iterate over each column and apply the function to fully standardise or revert the data:

```

7  # function takes column_name, applies standardisation formula to each row,
8  # then replaces data with standard values
9  def standardise(column_name):
10     global df, df2
11
12     # establish max and min values for given column
13     max = df[column_name].max()
14     min = df[column_name].min()
15
16     # save max and min values to excel
17     df2.at[0, column_name] = max
18     df2.at[1, column_name] = min
19
20     # compute standardised value at every row
21     for i, row in df[column_name].iteritems():
22         standard_value = 0.8*((row-min)/(max-min))+0.1
23         df.loc[i, column_name] = standard_value
24
25 # function takes column_name, applies inverse standardisation formula
26 # to each row, then reverts data with original values
27 def destandardise(column_name):
28     global df, df2
29
30     # establish max and min values for given column
31     max = df2.loc[0, column_name]
32     min = df2.loc[1, column_name]
33
34     # compute destandardised value at every row
35     for i, standard_value in df[column_name].iteritems():
36         original_value = ((standard_value-0.1)/0.8)*(max-min)+min
37         df.loc[i, column_name] = original_value
38
39 # iterate over columns and apply function
40 for column_name in df:
41     destandardise(column_name)

```

Here we can observe a sample of the difference between standardised and original data:

AREA	BFIHOST	FARL	FPEXT	LDP	PROPWET	RMED-1D	SAAR	Index flood		AREA	BFIHOST	FARL	FPEXT	LDP	PROPWET	RMED-1D	SAAR	Index flood
1800.92	0.471	0.959	0.0819	152.3	0.43	38.8	1110	183.0255		0.41392	0.382776	0.807606	0.357293	0.596165	0.366667	0.419777	0.292838	0.247382
31.4	0.514	0.998	0.048	14.18	0.4	35.2	922	9.581		0.105194	0.426992	0.895493	0.247717	0.13957	0.328571	0.379666	0.227162	0.107607
130.1	0.653	0.985	0.1461	30.25	0.28	27.5	596	3.815		0.122414	0.569923	0.866197	0.564808	0.192694	0.17619	0.293872	0.113275	0.10296
344.36	0.461	0.909	0.0258	42.68	0.41	38.5	1170	80.671		0.159796	0.372494	0.69493	0.17596	0.233785	0.34127	0.416435	0.313799	0.164897
710.96	0.458	0.995	0.1129	61.41	0.57	36.4	1153	200.849		0.223756	0.369409	0.888732	0.457495	0.295702	0.544444	0.393036	0.30786	0.261746
67.25	0.36	0.961	0.1069	22.87	0.35	36.3	820	19.9025		0.111449	0.268638	0.812113	0.438101	0.168298	0.265079	0.391922	0.191528	0.115925
851.65	0.616	0.966	0.1636	53.5	0.34	29.2	694	38.238		0.248302	0.531877	0.82338	0.621374	0.269554	0.252381	0.312813	0.147511	0.130701
70.46	0.415	1	0.0618	26.99	0.3	31.5	622	10.044		0.112009	0.325193	0.9	0.292323	0.181917	0.201587	0.33844	0.122358	0.10798
73.06	0.402	1	0.0287	23.15	0.65	52.8	1845	56.542		0.112462	0.311825	0.9	0.185333	0.169223	0.646032	0.575766	0.549607	0.145452
661.92	0.438	0.844	0.0746	67.37	0.63	54.5	1810	203.101		0.2152	0.348843	0.548451	0.333697	0.315405	0.620635	0.594708	0.53738	0.263561
2172.36	0.318	0.961	0.0504	94.29	0.51	35.6	1016	880.582		0.478725	0.22545	0.812113	0.255475	0.404397	0.468254	0.384123	0.26	0.809529
204.57	0.41	0.95	0.1055	32.05	0.3	33.8	664	15.28		0.135407	0.320051	0.787324	0.433576	0.198645	0.201587	0.364067	0.137031	0.112199
183.79	0.86	0.969	0.0537	38.23	0.36	37	942	8.17		0.131781	0.782776	0.830141	0.266141	0.219074	0.277778	0.399721	0.234148	0.10647
41.01	0.315	0.998	0.025	12.1	0.62	38.1	1129	29.912		0.106871	0.222365	0.895493	0.173374	0.132694	0.607937	0.411978	0.299476	0.123991

Constructing the Artificial Neural Network:

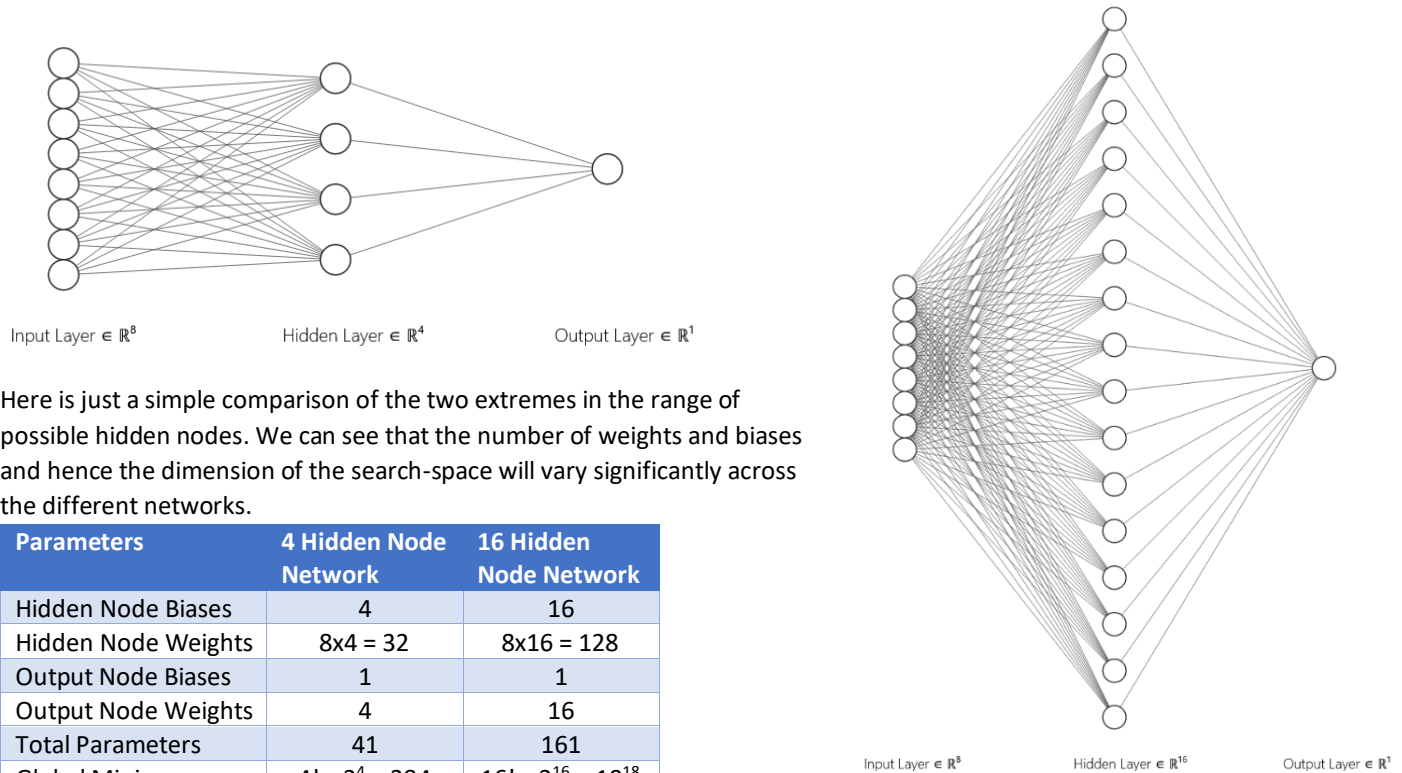
Number of Hidden Nodes:

We can see from the dataset that we have 8 predictors and one single predictand. Therefore, we know that our Neural Network will contain 8 input nodes, and one output node. By suggestion in the lecture videos, I will only construct a single hidden layer, since it has been said in those videos that for simple problems, i.e., problems with a single predictand, a single hidden layer is said to be sufficient.

It is claimed in the lecture slides and videos that:

Number of hidden nodes $\in [\frac{n}{2}, 2n]$, where n is number of predictands

Therefore, the number of hidden nodes in our Neural Network will range from 4 to 16 hidden nodes, with the final number depending on whichever network performs the best on our validation set, i.e., what number produces the smallest error.



Here is just a simple comparison of the two extremes in the range of possible hidden nodes. We can see that the number of weights and biases and hence the dimension of the search-space will vary significantly across the different networks.

Parameters	4 Hidden Node Network	16 Hidden Node Network
Hidden Node Biases	4	16
Hidden Node Weights	$8 \times 4 = 32$	$8 \times 16 = 128$
Output Node Biases	1	1
Output Node Weights	4	16
Total Parameters	41	161
Global Minima	$4! \times 2^4 = 384$	$16! \times 2^{16} = 10^{18}$

Initialise weights, biases randomly within: $[-\frac{2}{n}, \frac{2}{n}]$, where n is number of inputs

For the sake of simplicity and ease of testing, I will generate a random set of 161 numbers within that range, then each multi-layer perceptron will take its initial weights and biases from this pool. The code for this is trivial when using Pandas for Python:

```

1 import pandas as pd
2 import random
3
4 df = pd.read_excel("C:/Users/jakub/OneDrive/Documents/NeuralNetworks/Data/Random.xlsx", na_values=[
5
6 def generate_values():
7     # iterate over max number of parameters for Largest Neural Network
8     # generate random values and write to dataframe
9     for i in range(1, 168):
10         df.loc[i, "Input-Hidden Weights"] = random.uniform(-0.25, 0.25)
11     for j in range(1, 16):
12         df.loc[j, "Hidden-Output Weights"] = random.uniform(-0.25, 0.25)
13     for k in range(1, 16):
14         df.loc[k, "Hidden Biases"] = random.uniform(-0.25, 0.25)
15     df.loc[1, "Output Biases"] = random.uniform(-0.25, 0.25)
16
17     # write dataframe to excel sheet
18     df.to_excel('C:/Users/jakub/OneDrive/Documents/NeuralNetworks/Data/Random.xlsx', index=False)
19
20 generate_values()

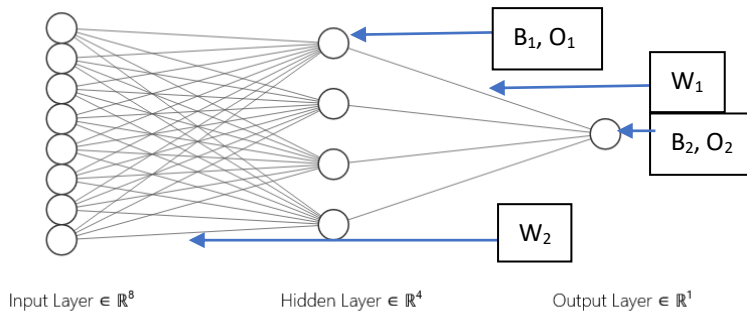
```

	Input-Hidden Weights	Hidden-Output Weights	Hidden Biases	Output Biases
1				0.231134004
2	-0.133265422	-0.057081483	0.086069648	
3	-0.241899775	0.12922933	-0.089173451	
4	0.08352402	0.100670083	0.0044238	
5	0.020776854	-0.155016025	-0.032769706	
6	-0.185084724	0.000589532	0.031609252	
7	-0.18635316	0.123996748	0.231451287	
8	0.089340486	0.010675384	0.140843442	
9	0.082139796	0.135198709	-0.118750376	
10	0.057214483	-0.170513706	0.122571662	
11	0.099951546	0.106295379	-0.145887099	
12	0.0931933	0.002832445	-0.219709969	
13	-0.248043757	0.024999453	-0.196111016	
14	0.043978307	-0.144213269	0.1884602	
15	0.10905397	0.247707793	-0.163437806	
16	0.140988611	0.145933888	-0.199132499	
17	0.061729828			

Coding the Multi-Layer Perceptron:

To code the Neural Network, I will construct a MLP class with multi-dimensional arrays to store weights, biases and outputs of neurons, and methods to initialise and modify those weights and biases accordingly. The three key arrays we will use are:

- Double outputs[layer][neuron]
- Double biases[layer][neuron]
- Double weights[layer][neuron][prevNeuron]



For examples on how outputs, biases are accessed for the example on the left:

$B_1 = \text{biases}[1][0]$, $B_2 = \text{biases}[2][0]$
 $O_1 = \text{outputs}[1][0]$, $O_2 = \text{outputs}[2][0]$
 $W_1 = \text{weights}[1][0][0]$, $W_2 = \text{weights}[0][3,7]$

Below is the constructor for the MLP class. As you can see the primary purpose of the constructor is to initialise the various arrays with appropriate sizes for each layer. The sizes themselves are determined by the number of input nodes, hidden nodes, and output nodes, which are taken as inputs for the construction of the MLP object. Weights and biases are taken from the excel file containing initial values using a rudimentary class called Extractor that reads the Excel file.

```

38 public MLP(int INPUT_SIZE, int HIDDEN_SIZE, int OUTPUT_SIZE, double learningParameter, double[][] inputs, double[][] validationData){
39     // set learning parameter
40     this.learningParam = learningParameter;
41
42     // define number of neurons in each layer
43     this.INPUT_SIZE = INPUT_SIZE;
44     this.HIDDEN_SIZE = HIDDEN_SIZE;
45     this.OUTPUT_SIZE = OUTPUT_SIZE;
46
47     // initialise number of layers
48     this.outputs = new double[3][];
49     this.biases = new double[2][];
50     this.weights = new double[2][][];
51
52     // initialise number of outputs for each layer
53     this.outputs[0] = new double[INPUT_SIZE];
54     this.outputs[1] = new double[HIDDEN_SIZE];
55     this.outputs[2] = new double[OUTPUT_SIZE];
56
57     // initialise biases for hidden and output layers
58     this.biases[0] = extractor.getWeightsBiases( layer: 2, start: 1, HIDDEN_SIZE);
59     this.biases[1] = extractor.getWeightsBiases( layer: 3, start: 1, OUTPUT_SIZE);
60
61     // initialise weights between layers
62     this.weights[0] = new double[HIDDEN_SIZE][INPUT_SIZE];
63     this.weights[1] = new double[OUTPUT_SIZE][HIDDEN_SIZE];
64     for(int i=0; i<HIDDEN_SIZE; i++)
65     {
66         int row = (8*i)+1;
67         weights[0][i] = extractor.getWeightsBiases( layer: 0, row, end: row+7);
68     }
69     weights[1][0] = extractor.getWeightsBiases( layer: 1, start: 1, HIDDEN_SIZE);
70
71     // define data for MLP object
72     this.data = inputs;
73     this.validationData = validationData;
74 }

```

The next step is to code the algorithm computing the forward pass. Fundamentally, this algorithm is a process of taking a sequence of predictands, setting the first layer of outputs to the predictands (inputs for the forward pass can be taken as outputs of the first layer of nodes.). We then take a node, get the bias from that node, then get the sum of products of weights and outputs leading to that node. Add the bias to that sum, then apply the activation function. Take that as the output for that node and repeat the process.

```

76 private double activation(double sum){
77     return 1/(1+Math.exp(-sum));
78 }
79
80 private double forwardPass(double[] predictands){
81     // equate outputs of input nodes to given predictands
82     this.outputs[0] = predictands;
83     for(int i=0; i<biases.length; i++){
84         for(int j=0; j<biases[i].length; j++){
85             {
86                 // get bias at given node
87                 double bias = biases[i][j];
88                 double sum = 0;
89
90                 // iterate over all weights and outputs leading to bias node, sum then apply activation function
91                 for(int k=0; k<weights[i][j].length; k++){
92                     double weight = weights[i][j][k];
93                     double prevOutput = outputs[i][k];
94                     sum += weight*prevOutput;
95                 }
96                 outputs[i+1][j] = activation( sum+sum+bias);
97             }
98         }
99     }
    return outputs[outputs.length-1][0];

```

Now this algorithm functions for a single output node at the moment, which is somewhat problematic as the MLP class is supposed to take any arbitrary amount of input, hidden and outputs nodes. However, for the purpose of this problem, the algorithm will suffice, so I will not be making the necessary modifications.

After the forward pass, we obtain the prediction from the network. In order to backpropagate, we need to set the necessary δ_j 's at output and hidden nodes. Since we are using the sigmoid activation, the derivative and δ_j 's are as follows (taken from slides):

$$f'(S_j) = u_j(1-u_j)$$

$$\delta_j = \begin{cases} (C-u_o) f'(S_o) \\ w_{j,o} \delta_o f'(S_j) \end{cases}$$

```

120 private void backPropagate(double actual, double prediction) {
121     // initialise deltaJ
122     double[][] deltaJs = new double[2][];
123     deltaJs[0] = new double[HIDDEN_SIZE];
124     deltaJs[1] = new double[1];
125
126     // set deltaJ for output nodes, using derivative of sigmoid function
127     double derivative = prediction*(1-prediction);
128     deltaJs[1][0] = (actual-prediction)*derivative;
129
130     // set deltaJ for hidden nodes
131     for(int i=0; i<HIDDEN_SIZE; i++){
132         double output = outputs[1][i];
133         derivative = output*(1-output);
134         deltaJs[0][i] = weights[1][0][i]*deltaJs[1][0]*derivative;
135     }
136     this.updateWeightsBiases(deltaJs);
137 }

```

The algorithm takes the predicted value derived from the forward pass algorithm, compares it with the actual value, and computes the δ_j 's accordingly. You may note that once again this algorithm is dependant on there being a single output node and a single hidden layer, as the δ_j for hidden nodes are dependent on δ_o which is the δ_j for the output node. Once again, it suffices for the problem we have, and hence I will keep it in its current state.

After computation of δ_j 's has been completed for all hidden and output nodes, it remains to adjust all the weights and biases according to those δ_j 's. This is done accordingly to the following formula in the slides:

$$w_{i,j}^* = w_{i,j} + \rho \delta_j u_i$$

Where $w_{i,j}^*$ is the new weight/ bias, $w_{i,j}$ is the current weight/bias, ρ is the learning parameter, U_i is the output out of that node.

```

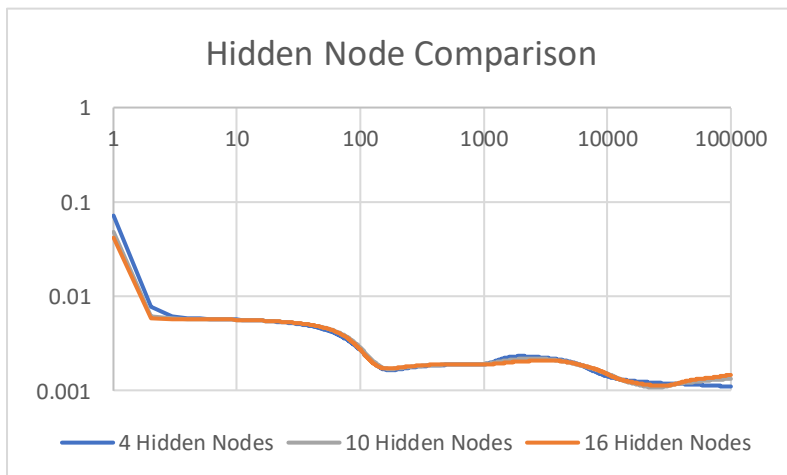
101 private void updateWeightsBiases(double[][] deltaJs){
102     // iterate over weights and biases
103     for(int layer=0; layer<2; layer++){
104         for(int neuron=0; neuron<weights[layer].length; neuron++){
105             // get deltaJ at the given neuron
106             double deltaJ = deltaJs[layer][neuron];
107
108             // get and update bias at neuron
109             double bias = biases[layer][neuron];
110             biases[layer][neuron] = bias + this.learningParam*deltaJ* outputs[layer+1][neuron];
111
112             for(int prevNeuron=0; prevNeuron<weights[layer][neuron].length; prevNeuron++){
113                 // get and update weight moving out of neuron
114                 double weight = weights[layer][neuron][prevNeuron];
115                 weights[layer][neuron][prevNeuron] = weight + this.learningParam*deltaJ*outputs[layer][prevNeuron];
116             }
117         }
118     }

```

We have everything required to run an epoch of the MLP. Therefore, I will construct a public method, called `runNetwork()`, that takes a given number of epochs as input, and calls the functions previously defined iteratively, storing errors for the training and validation sets.

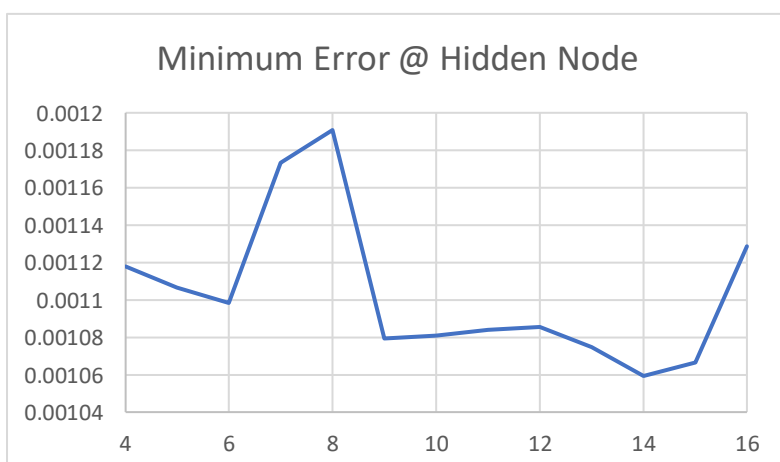
Analysis of the Base Algorithm:

Now we have everything required in order to construct multiple MLP's, of varying number of hidden nodes, and decide on which is the most optimal. To do this I will write the validation errors into excel and plot them in a log-log scale. I will use a log-log scale to space out the lines in the graphs to allow for clearer analysis.



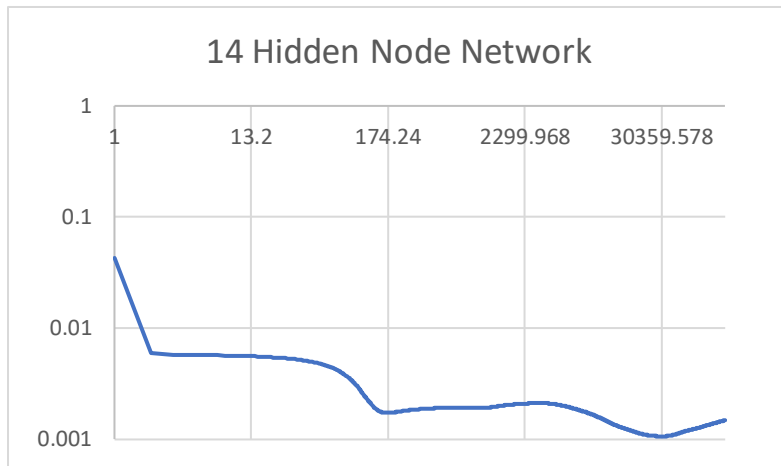
It is immediately clear that the number of hidden nodes does not play any major role in the error function generated from the validation data. However, we still need to decide which is the most optimal, and since there are no clear differences in the graphs, even with a log-log plot, I will simply select the number of nodes that produces the smallest error at any epoch.

This will also show us how many epochs to run for the base algorithm to avoid over-fitting.



We can see that this graph is far more informative than the previous. It is clear here that the number of hidden nodes that produces the lowest error is 14.

Next, I select the error column produced by a neural network with 14 hidden and use a simple Excel formula to locate the row and hence epoch that produces this lowest error. The result I get is that training the network for 30,000 epoch produces the lowest error.



I have plotted the network with 14 hidden nodes, with a $\log_{10}$ y-axis and $\log_{13.2}$ x-axis to further illustrate the fact that the lowest error is produced at roughly 30,000 training epochs.

Now that we have located the most optimal network configuration for the base algorithm with no improvements, we should analyse the predictions made by this network for the testing dataset and compare it to the actual values. This way we can get a rough idea of how well the network actually performs in a real-world scenario.

1	AREA	BFIHOST	FARL	FPEXT	LDP	PROPWET	RMED-ID	SAAR	index floor	Prediction
2	11.76	0.597	0.923	0.0183	7.41	0.38	41.4	1030	1.234	-8.43334
3	20.38	0.539	0.978	0.0028	12.14	0.54	50.9	1485	7.7565	5.802263
4	198.07	0.553	0.999	0.0295	32.69	0.48	50.8	1555	111.322	88.42543
5	125.92	0.553	0.999	0.0382	28.39	0.49	37.5	962	26.7795	28.06058
6	92.61	0.536	0.989	0.0297	22.2	0.43	34.2	762	17.979	15.91879
7	883.12	0.528	0.976	0.0746	87.48	0.42	34.3	936	119.094	126.452
8	203.65	0.562	0.977	0.0537	40.88	0.36	36.2	964	43.723	23.7088
9	77.73	0.386	0.947	0.0635	25.81	0.5	37.8	1181	16.774	18.83236
10	194.15	0.381	0.994	0.1067	44.61	0.34	33.3	726	43.54	21.70074
11	1283.4	0.426	0.96	0.0412	84.45	0.53	44.7	1386	529.327	524.8068
12	89.42	0.451	1	0.0172	23.5	0.53	60	2038	101.89	69.73608
13	148.7	0.538	0.994	0.1466	33.49	0.35	28.8	729	15.362	1.078064
14	307.14	0.489	0.986	0.0503	43.22	0.43	34.7	713	57.803	53.07092
15	140.38	0.493	0.994	0.0688	40	0.31	28.4	579	13.836	2.523913
16	938.48	0.452	0.966	0.0616	85.17	0.61	38.5	1204	247.7385	282.6579
17	133.52	0.551	0.965	0.0329	37.06	0.49	38	906	36.8565	19.45737
18	188.05	0.654	0.954	0.19	34.67	0.4	29	739	10.653	7.337997
19	117.35	0.422	0.97	0.0458	28.01	0.36	39.4	899	27.434	22.85898
20	15.85	0.959	1	0.0305	7.33	0.32	33.9	757	0.142	-2.33904
21	801.65	0.437	0.943	0.1041	66.98	0.3	30.2	648	62.661	96.5354
22	139.27	0.436	0.997	0.0284	33.73	0.57	32.7	915	71.038	49.82477
23	78.54	0.28	0.998	0.0224	17.99	0.63	39.2	1156	53.15	55.23046
24	32.34	0.345	0.645	0.0154	10.6	0.71	70.2	2438	12.811	-47.3827
25	109.39	0.512	0.896	0.0293	28.24	0.68	43.8	1244	69.8325	34.64519
26	149.24	0.404	0.743	0.0382	23.62	0.67	62.8	2150	59.096	37.39654
27	322.97	0.27	0.995	0.0303	35.83	0.64	41.4	1331	237.306	239.0023
28	1149.07	0.442	0.836	0.0373	89.04	0.7	49.4	1950	354.832	343.3047
29	74.13	0.362	0.999	0.0106	17.37	0.59	34.8	895	25.0555	40.33438
30	55.72	0.82	0.958	0.0247	13.54	0.29	35.4	698	3.084	3.685367

Here we can observe the predictions made by the current best configuration of the network. All values have been de-standardised for clarity. My judgement thus far is that the predictions are mostly quite sensible with the exception of negative values, which will be handled later. The network seems to perform best when the prediction is large, and worse with small values. That said, there is obviously a lot of room for improvement, which is expected of the base algorithm.

The first improvement we will look at is momentum, so that we can converge to an optimal solution in a smaller number of epochs, or even find new minima (when max epochs are equated).

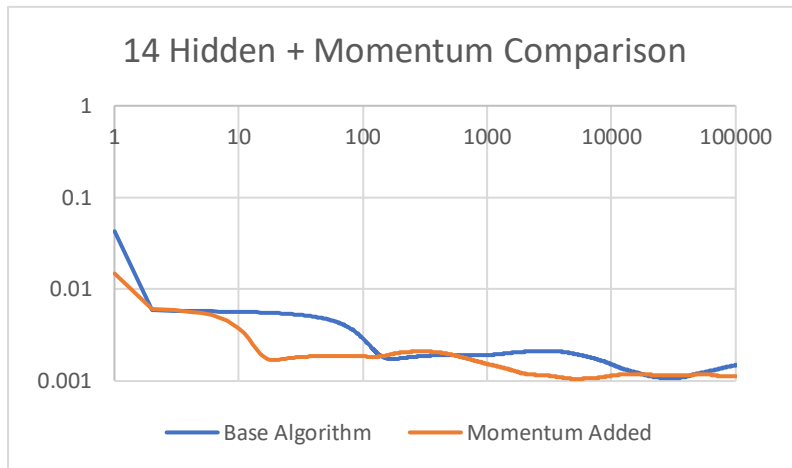
Implementation of Momentum Improvement:

Momentum is an easy improvement to implement as it only involves adding an additional term to the algorithm that adjusts weights and biases at given node. This term is the product of an α co-efficient, which I will set to 0.9 according to the lecture slides, and the difference between the new weight/ bias and old weight/ bias. The formula is below:

Gradient descent with momentum: $W_{i,j}^* = W_{i,j} + \rho \delta_j v_i$

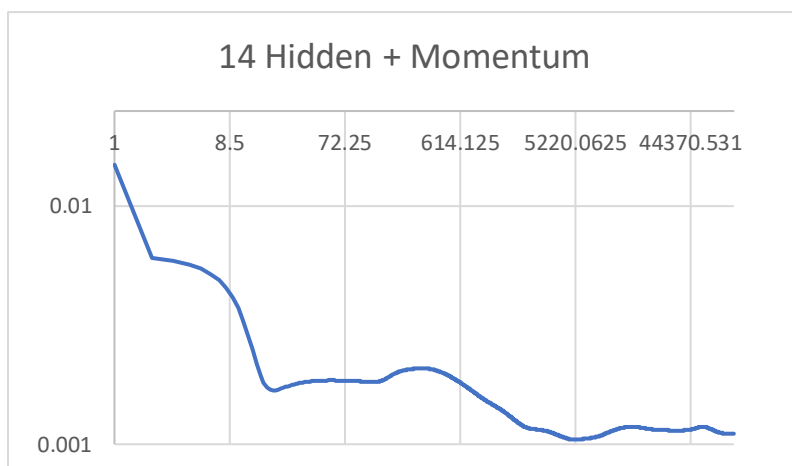
$\Delta W_{i,j} = W_{i,j}^* - W_{i,j}$, where $W_{i,j}^*, W_{i,j}$ are new and old weights respectively

The code for this is rudimentary, it only involves storing changes in weights and biases and adding the term, provided momentum is set to true. Now let us observe the difference in error function for momentum and the base algorithm at our current configuration:



Observe that not only do we converge to an optimal solution with less epochs than the base algorithm, but we also have a smaller overall error across all epochs. This is a clear and measured improvement to our neural network. Therefore, all subsequent networks will have momentum activated by default.

Now let us analyse the momentum graph more closely.



Observe that the number of epochs required to find out minima is roughly 5000, which means that our algorithm found the most optimal solution with 500% less epochs!

That said, our minimal error has not decreased by any substantial margin, and hence this model is not likely to present any major improvement when tested against our test dataset.

We must therefore continue adding improvements.

Implementation of the Bold Driver Improvement:

To implement bold driver, we simply need to check the errors array after every epoch, check if it has increased or decreased then apply a multiplier, so long as it stays within the range of [0.01, 0.5] mentioned in the lecture notes. The code required to do this is once more rudimentary, but I will showcase it regardless:

```

218 // bold driver check
219 if(this.boldDriver && i>0){
220     // if error smaller and adjustment within range
221     if(errors[i] < errors[i-1] && this.learningParam*1.05<0.5){
222         this.learningParam *= 1.05;
223     }
224     // else if error larger and adjustment within range
225     else if(errors[i] > errors[i-1] && this.learningParam*0.7>0.01){
226         this.learningParam *= 0.7;
227     }

```

Note that here we are comparing values in the training error function, not the validation error function. I will also try using the validation error function as a comparator, and decide on which produces the most favorable results, i.e. finds the smaller minima.

